12: Virtual Memory

Engr 315: Hardware / Software Codesign

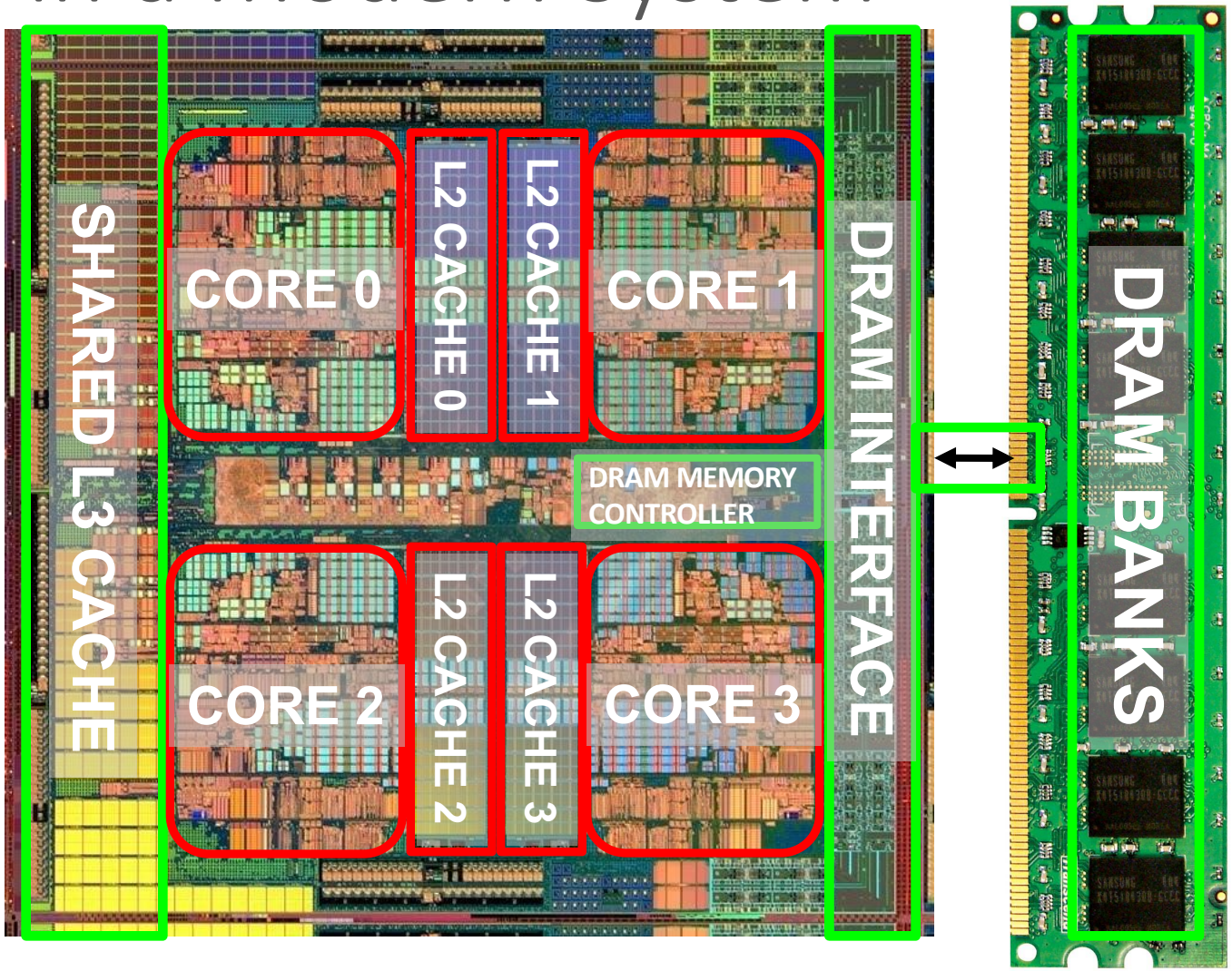
Andrew Lukefahr
Indiana University

Some material taken from EECS370 at U. of Michigan

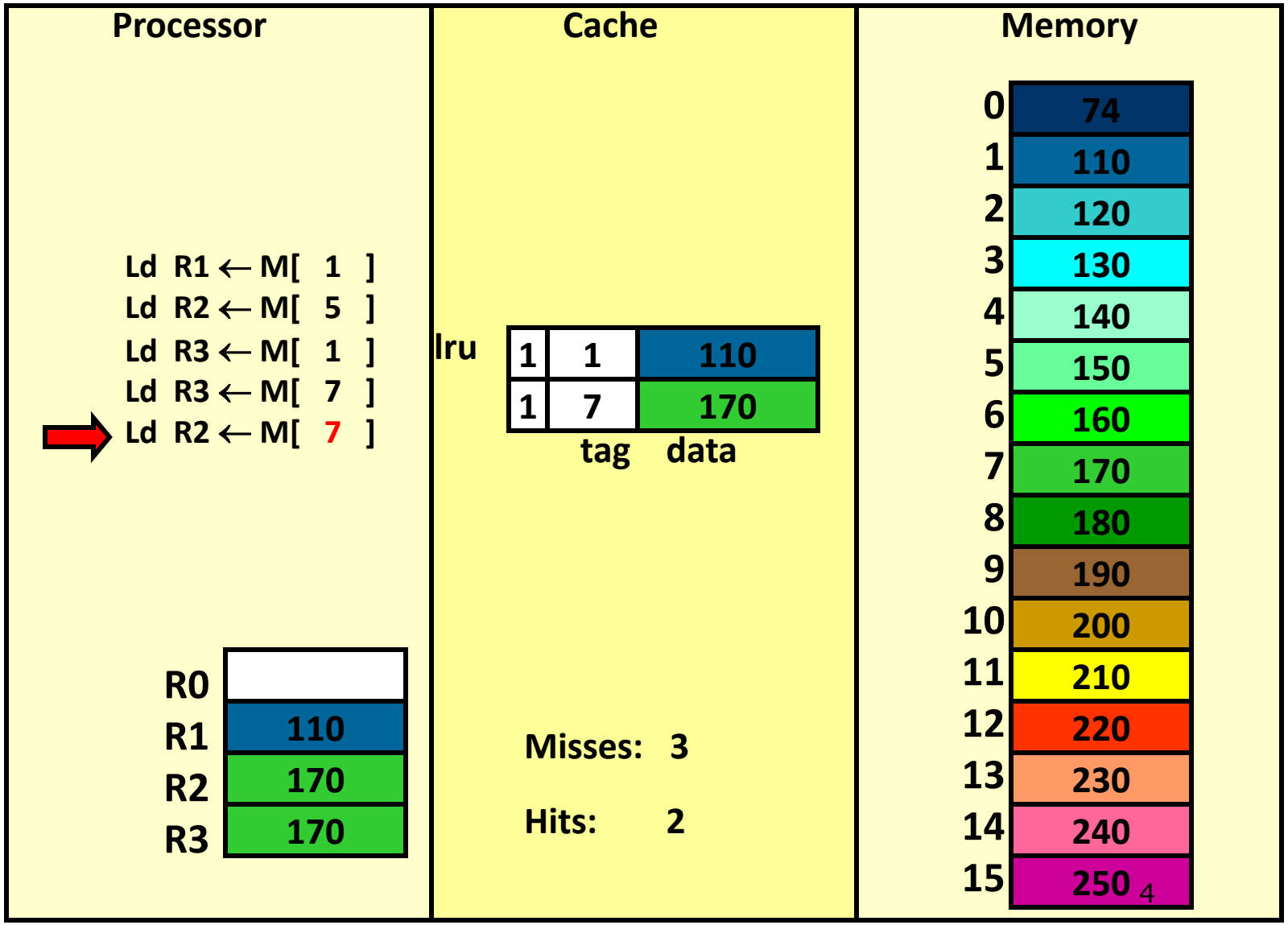
Announcements

- P3: Extended a week.
- P4: Count the number of binary 1's in a file.
  - Going to involve Linux on the Pynq.
  - I still need to figure out *how* to do that.
- Exam next Wednesday (3/3)
  - Review on Monday

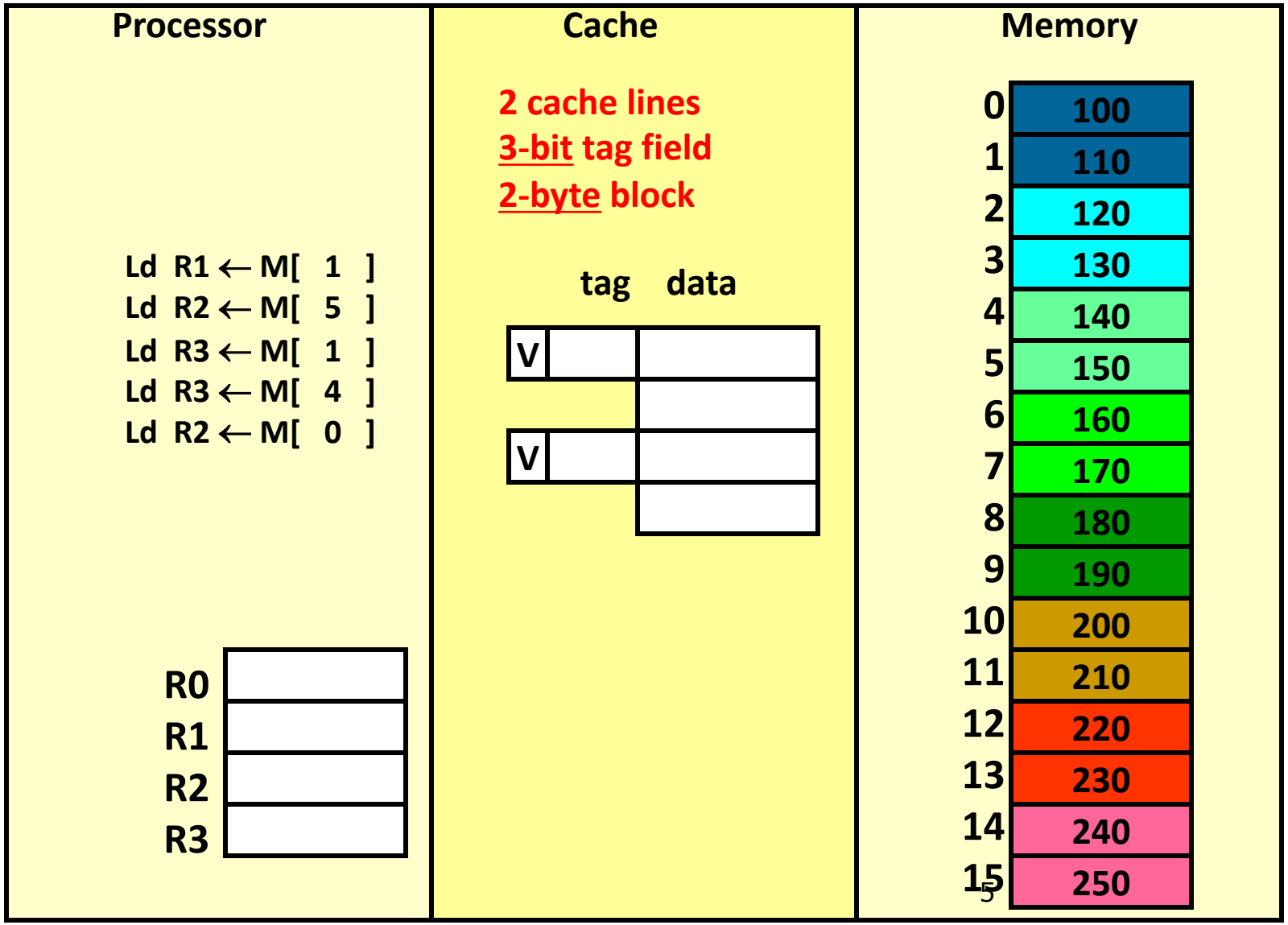
Memory in a Modern System



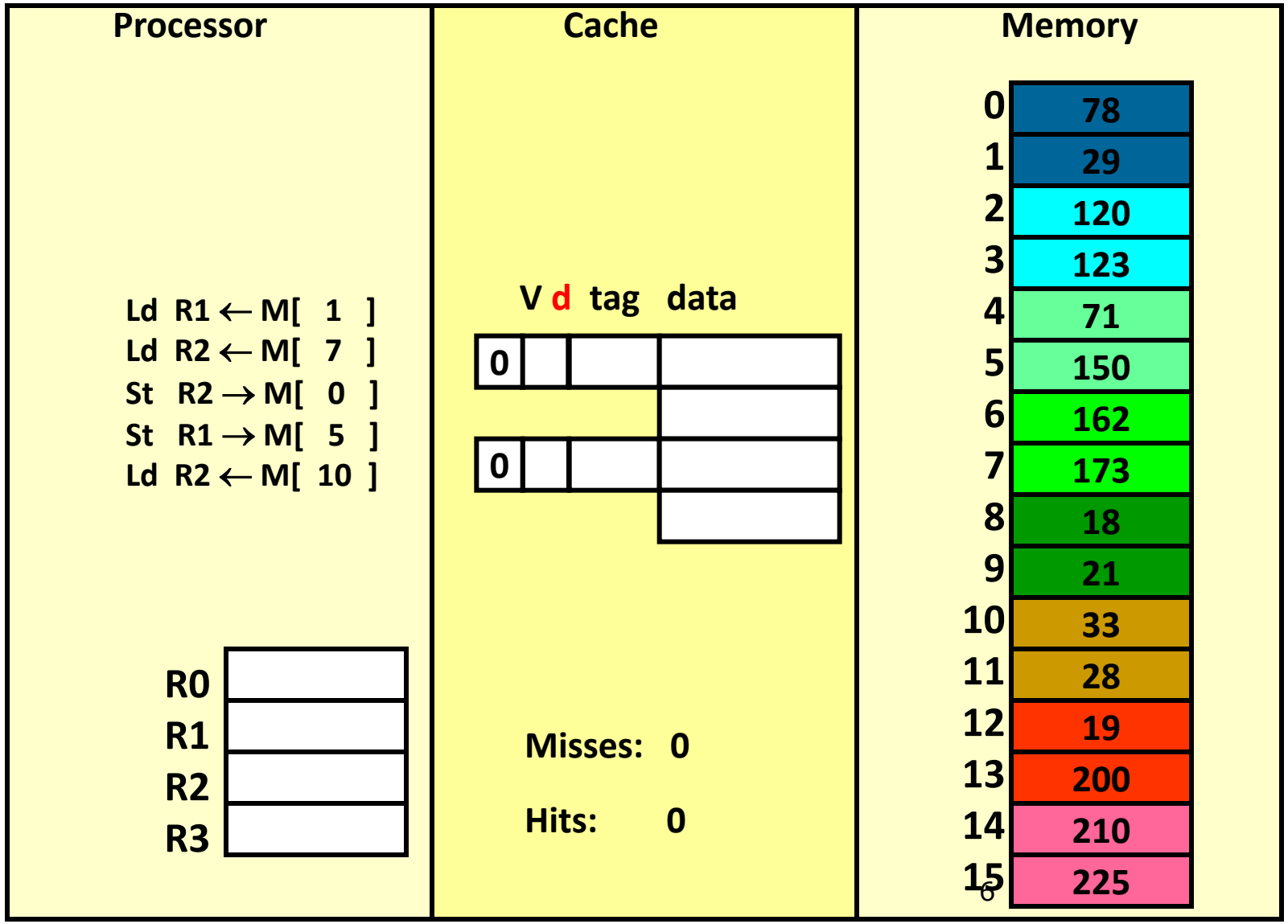
Last Time: A Very Simple Memory System



Block size for caches



Handling stores (write-back)



Class Problem 2

- Consider the following cache:
  32-bit memory addresses, byte addressable, 64KB cache
  64B cache block size, write-allocate, write-back, ~~*fully associative*~~

  This cache will need 512 kilobits for the data area (64 kilobytes times 8 bits per byte). Note that in this context, 1 kilobyte = 1024 bytes (NOT 1000 bytes!) Besides the actual cached data, this cache will need other storage. Consider tags, valid bits, dirty bits, bits to keep track of LRU, and anything else that you think is necessary.

- How many additional bits (not counting the data) will be needed to implement this cache ?

The Problem(s) with memory

1. 32-bit address vs. 512MB of RAM

2. How to separate multiple applications?

32-bit address vs. 512MB of RAM

- The Pynq has 512 MB of memory.
  - 512MB = 2^29 Bytes
- The CPU uses 32-bit addresses
  - 0x00000000 to 0xFFFFFFFF
  - 2^32 Bytes = 4 GB
- Where is the other 3.5 GB?

Separating multiple applications?

- A) What if two applications want to use the same memory address?
- B) How do I prevent your application from modifying my memory?

Two application test..

```c
#include <stdio.h>
#include <stdlib.h>

volatile int avalue = 2;

int main ()
{

    while (avalue == 2) { ; }
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>

volatile int avalue = 3;

int main ()
{

    while (avalue == 3) { ; }
    return 0;
}
```

Two application test..

```
gcc -g -O0 test_1.c -o test_1.out
objdump -DSs test_1.out > test_1.dis
gcc -g -O0 test_2.c -o test_2.out
objdump -DSs test_2.out > test_2.dis

vi test_1.dis test_2.dis
```

Two application test..

```
00011008 <avalue>:
volatile int avalue = 2;
   11008:       00000002        andeq   r0, r0, r2
```

```
00011008 <avalue>:
volatile int avalue = 3;
   11008:       00000003        andeq   r0, r0, r3
```

Two application test..

```
./test_1.out &
./test_2.out &

top
```

Q: Why didn’t they clobber each other?

Q: Why didn’t they clobber each other?

- A: Linux uses “Virtual Memory”

Solution 3: Virtual memory

- Build new hardware and software that automatically translates each memory reference from a

virtual address

(which the programmer sees as a huge array of bytes)

to a

physical address

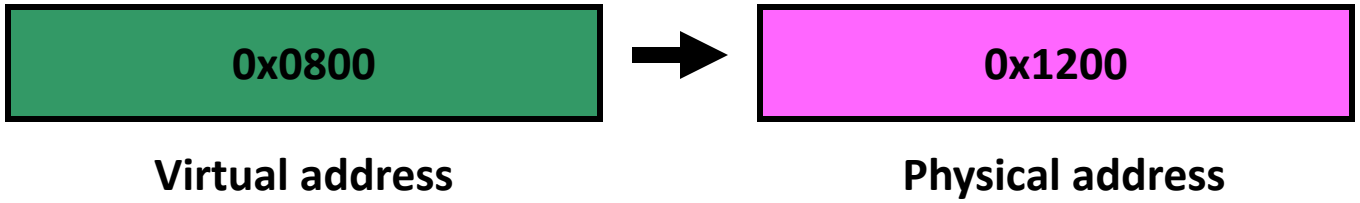
(which the hardware uses to identify where the storage actually resides)

Basics of Virtual Memory

- Any time you see the word virtual in computer science and architecture it means "using a level of indirection"

- Virtual memory hardware changes the virtual address the programmer sees into the physical one the memory chips see

test_1 →



test_2 → 0x0800 → 0x4C00

Virtual Memory View

- Virtual memory lets the programmer address a memory array larger than the DRAM available on a particular computer system

- Virtual memory enables multiple programs to share the physical memory without:
  - Knowing other programs exist (**transparency**)
  - Worrying about one program modifying the data contents of another (**protection**)

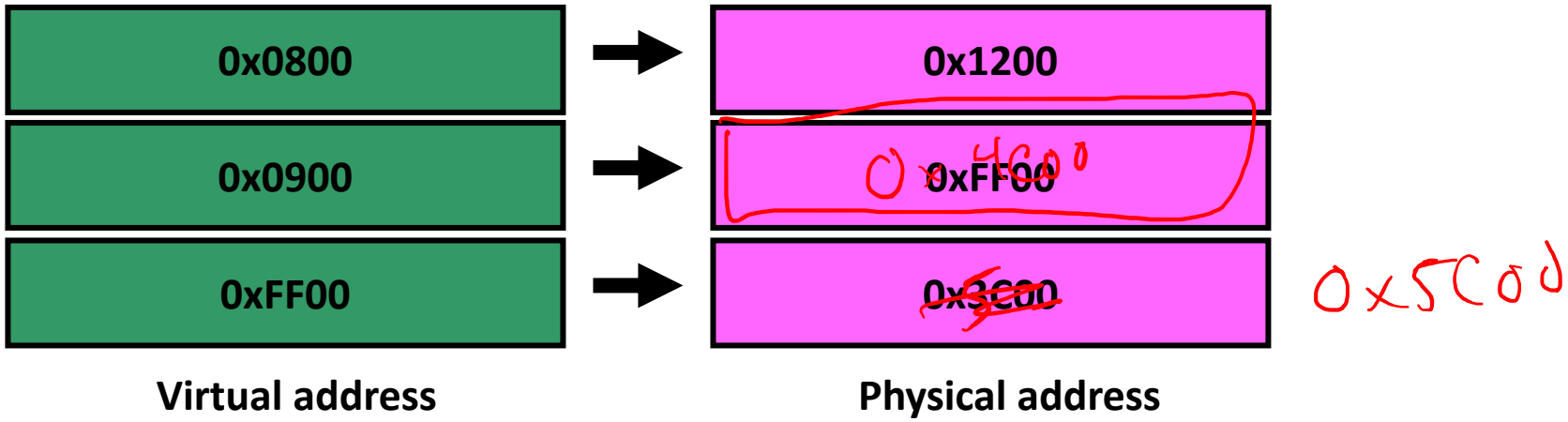
Managing virtual memory

- Managed by hardware logic ***and*** operating system software
  - Hardware for speed
  - Software for flexibility and because disk storage is controlled by the operating system

- The hardware must be designed to support VM

Translation Look-aside Buffers

- Buffer common Virtual->Physical translations in a **Translation Look-aside Buffer (TLB)**, a fast cache memory dedicated to storing a small subset of valid translations
- 16-512 entries common
- Generally has low miss rate (< 1%)



Translation Look-aside Buffer (TLB)

Virtual Address

32-bit

Virtual Base	Offset

20 bits

12-bits

Programmer

0x 12345 | abc

Virtual Base

v	~~tag~~	Physical Addr
V	12344	
V	12345	ABCDE
V	12346	

0xABCDE | ABC

32-bits

Physical Base	Offset
20	12

Physical Address

Where is the TLB lookup?

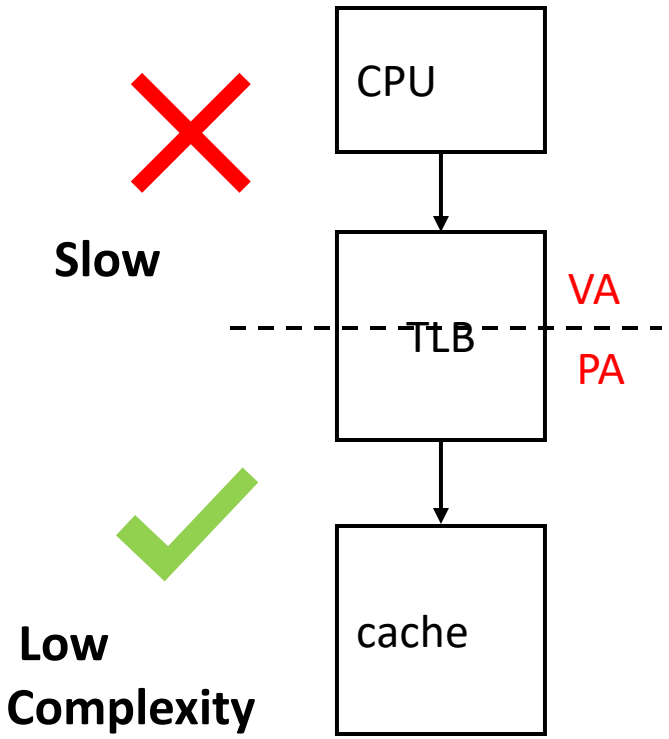
- We put the TLB lookup in the pipeline after the virtual address is calculated and before the memory reference is performed
  - This may be before or during the data cache access
  - Without a TLB hit we need to perform the translation during the memory stage of the pipeline

Caches in Systems with VM

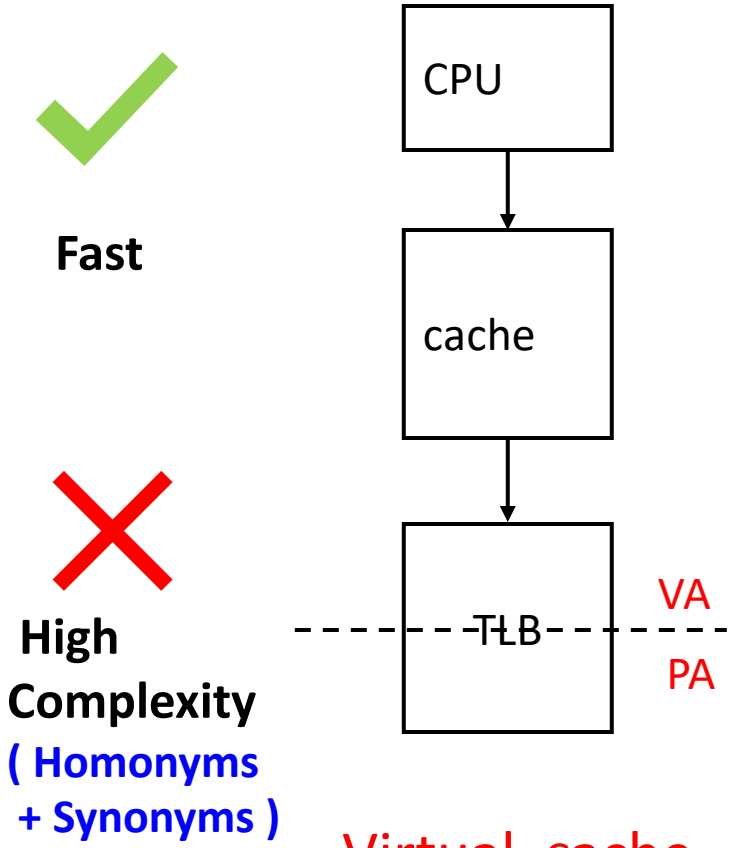
- VM systems give us two different addresses: virtual and physical

- Which address should we use to access the data cache?
  - Physical address (after VM translations)
    - Delayed access
  - Virtual address (before VM translation)
    - Faster access
    - More complex

Cache & VM Organization



Physical cache



Virtual cache

Adding TLBs

Processor

1 = 0001

✓ Ld R1 ← M[1]
✓ Ld R2 ← M[7]
St R2 → M[0]
St R1 → M[6]
Ld R2 ← M[10]

R0	
R1	21
R2	29
R3	

TLB

2-byte "page"

Address [3] [1]
tag offset

V	Tag	Physical
1	0	4
1	3	0

0001 → 1001
0111 → 0001
0000 → 1000
0110 → 0000
1010 → 0

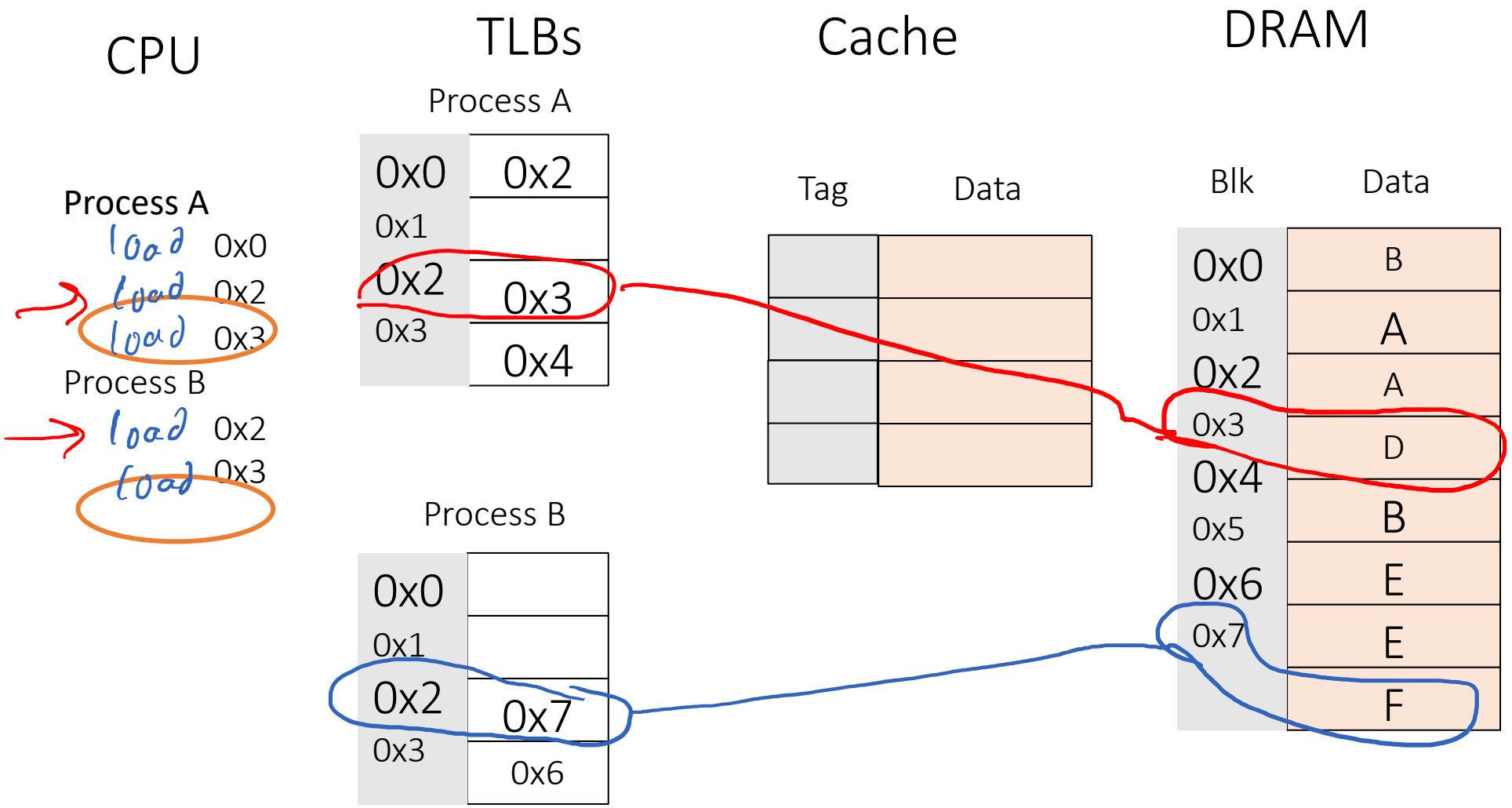
Cache (Physical)

V	d	tag	data
1	0	100	18
			21
1	0	000	78
			29

Memory

	0	78
	1	29
	2	120
	3	123
0100	4	71
	5	150
	6	162
	7	173
1000	8	18
	9	21
	10	33
	11	28
	12	19
	13	200
	14	210
	15	225

Homonyms (Physically Addressed)



Synonyms (Physically Addressed)

CPU

Process A

0x0
0x2
0x3

Process B

0x2
0x3

TLBs

Process A

0x0	0x2
0x1	
0x2	0x3
0x3	0x4

Process B

0x0	
0x1	
0x2	0x4
0x3	0x5

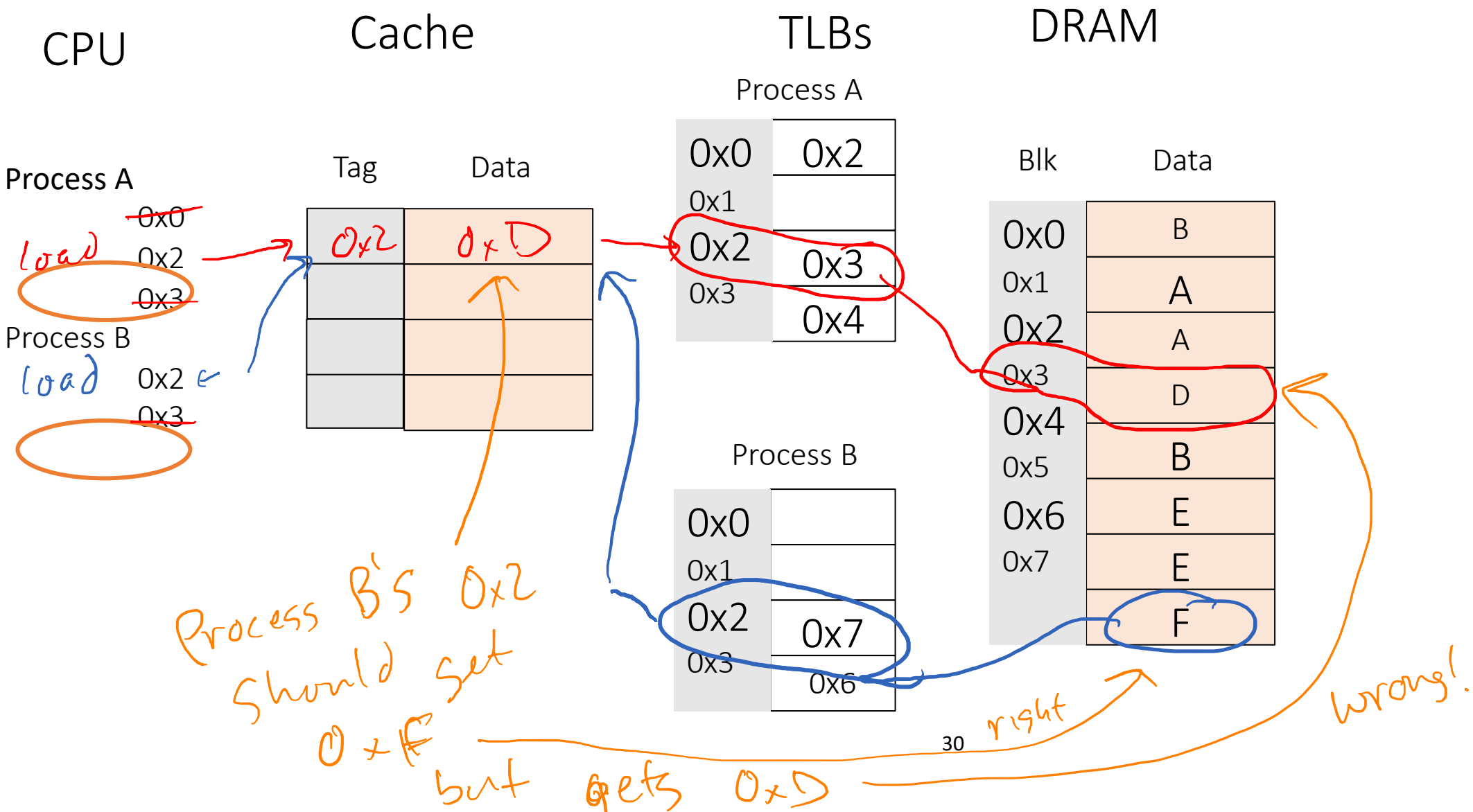
Cache

Tag	Data

DRAM

Blk	Data
0x0	B
0x1	A
0x2	A
0x3	D
0x4	B
0x5	E
0x6	E
0x7	F

Homonyms (Virtually Addressed)



Synonyms (Virtually Addressed)

CPU

Process A

0x0
0x2
0x3

Process B

0x2
0x3

Cache

Tag	Data

TLBs

Process A

0x0	0x2
0x1	
0x2	0x3
0x3	0x4

Process B

0x0	
0x1	
0x2	0x4
0x3	0x5

DRAM

Blk	Data
0x0	B
0x1	A
0x2	A
0x3	D
0x4	B
0x5	E
0x6	E
0x7	F

So how do we access the FPGA for P4?

- FPGA uses physical address
- CPU (w/Linux) uses virtual address

- Q: How do I talk to a physical address with Linux?

- A: Linux provides /dev/mem to help us!

Next Time

- Review

12: Virtual Memory

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University